

Implementing Directories

1 Basic Idea

Directory Implementation

A directory maps a human-readable file name to the information needed to locate that file's data on disk.

- Before a file can be read, it must be opened.
- During open, the operating system uses the path name to find the file's directory entry.
- The directory entry provides the information needed to locate the file's disk blocks.
- The exact location information depends on the allocation method used by the file system.

2 What a Directory Entry Stores

Allocation Method	Directory Entry May Store
Contiguous allocation	Disk address of the first block and file length.
Linked-list allocation	Disk address of the first block.
FAT-based allocation	Starting block number; the rest is found from the FAT.
I-node allocation	I-node number.

Main Function

The directory system maps the file's ASCII name to the internal information required to find the file data.

3 Where File Attributes Are Stored

Attributes Inside Directory Entries

- A directory can be stored as a list of fixed-size entries.
- Each entry contains a fixed-length file name.
- Each entry also contains file attributes.
- The entry may contain one or more disk addresses.
- This design is simple, but directory entries can become large.

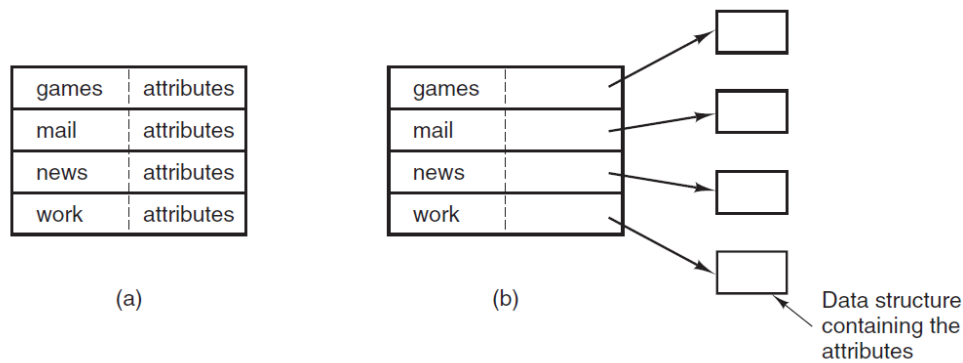


Figure 4-14. (a) A simple directory containing fixed-size entries with the disk addresses and attributes in the directory entry. (b) A directory in which each entry just refers to an i-node.

Figure 1: Two common ways to store directory entries and file attributes.

Attributes Inside I-Nodes

- In i-node-based systems, attributes are stored in the i-node.
- The directory entry only stores the file name and i-node number.
- This keeps directory entries shorter.
- It also separates naming from file metadata and block-location information.

4 Fixed-Length File Names

- Older systems often used short fixed-length names.
- MS-DOS used a 1–8 character base name plus an optional 1–3 character extension.
- UNIX Version 7 allowed names of 1–14 characters.
- A simple modern approach is to reserve a large fixed field, such as 255 characters, for every file name.
- This is easy to implement but wastes directory space because most names are much shorter than the maximum.

5 Variable-Length File Names

Method 1: In-Line Names

- Directory entries are no longer all the same size.
- Each entry begins with a fixed part, such as entry length and file attributes.
- The actual file name is stored directly after the fixed part.
- The name is usually terminated by a special character such as 0.
- Padding may be added so the next entry begins on a word boundary.

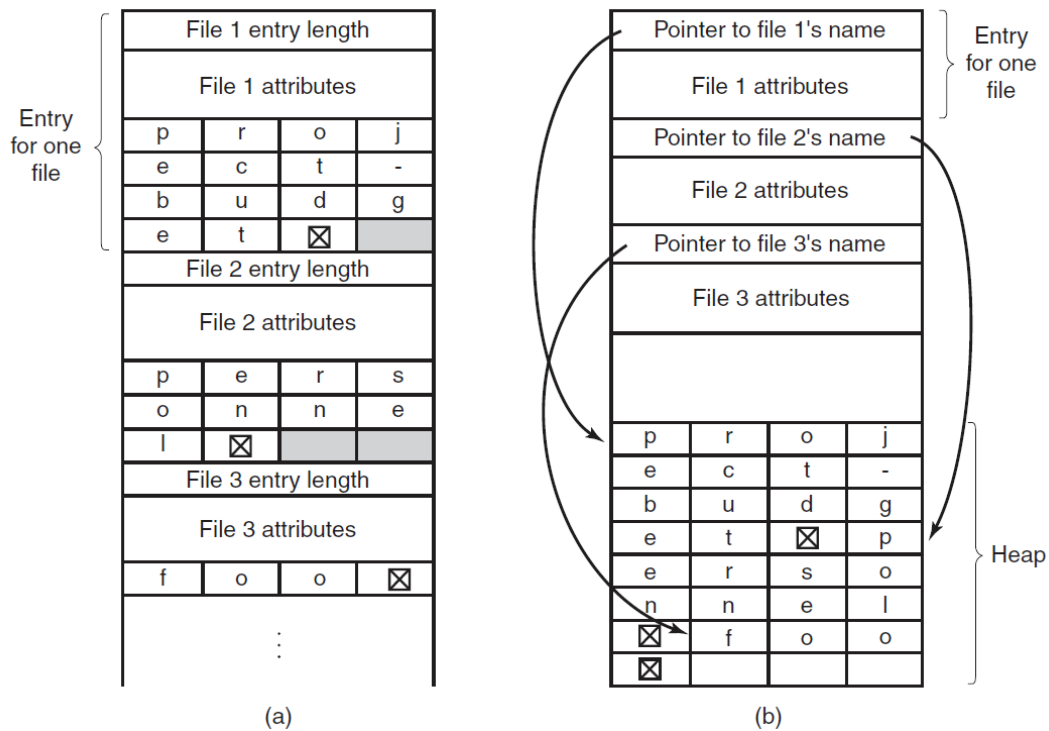


Figure 4-15. Two ways of handling long file names in a directory. (a) In-line. (b) In a heap.

Figure 2: Two ways of storing long variable-length file names in directories.

In-Line Name Problem

When a file is deleted, it leaves a variable-sized gap in the directory. A later file name may not fit into that gap without compacting the directory.

Method 2: Names in a Heap

- Directory entries remain fixed length.
- Each entry stores a pointer to the file name.
- The actual file names are stored together in a heap at the end of the directory.
- When a directory entry is removed, another fixed-size entry can reuse its place.
- No padding is required after file names because names do not need to start on word boundaries.

6 Comparing Long-Name Designs

Point	In-Line Names	Heap Names
Entry size	Variable length	Fixed length
Name location	Inside the directory entry	In a heap at the end
Deletion effect	Leaves variable-sized gaps	Fixed entries are easier to reuse
Padding	Often needed for word alignment	Usually not needed
Extra management	Directory compaction may be needed	Heap management is needed
Page-fault risk	Possible while reading long names	Also possible while reading heap names

7 Searching Directories

- The simplest directory lookup method is linear search.
- The operating system checks entries from the beginning until it finds the requested name.
- Linear search is acceptable for small directories.
- It becomes slow when a directory contains hundreds or thousands of files.

8 Hash Tables for Fast Lookup

Directory Hash Table

A directory can use a hash table to map file names quickly to directory entries.

- The file name is hashed to a value between 0 and $n - 1$, where n is the hash-table size.
- The hash value selects a slot in the table.
- If the slot is empty, the directory entry pointer can be stored there.
- If multiple names hash to the same slot, a linked list of entries is used.
- Lookup hashes the name and searches only the chain for that hash slot.

Hash Table Tradeoff

Hash tables make directory lookup faster, but they make directory administration more complex.

9 Caching Directory Lookups

- Another way to speed up lookup is to cache recent search results.
- Before scanning a directory, the file system first checks whether the name is already in the cache.

- If the name is cached, the file can be located immediately.
- Caching works best when a small set of files accounts for most lookups.

10 Key Points

- Directories translate file names into file-location information.
- Attributes may be stored directly in the directory entry or separately in an i-node.
- Fixed-length file names are simple but waste space.
- Variable-length file names save space but complicate directory management.
- Hashing and caching improve lookup performance for large or frequently used directories.

Final Point

Implementing directories is mainly about choosing how to store names, attributes, and location information so that file lookup is correct, compact, and fast enough for real workloads.